State

State · Order lifecycle NEW→PAID→SHIPPED with illegal guards · Build with Waqar

Today's Hook

- Problem: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Pattern: State — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



State: Order lifecycle NEW→PAID→SHIPPED with illegal guar

Real-World Scenario

- Running example: Order lifecycle NEW→PAID→SHIPPED with illegal guards
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;

import java.util.*;

public final class NaiveOrder {
  public String state="NEW";
  public void pay(){ state="PAID"; }
  public void ship(){ state="SHIPPED"; } // allows ship from NEW
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

enum Phase { NEW, PAID, SHIPPED, CANCELLED }
final class Order {
  private Phase phase=Phase.NEW;
  Phase phase(){ return phase; }
  void pay(){ if(phase!=Phase.NEW) throw new IllegalStateException("pay from "+phase); phase=Phase.PAID; }
  void ship(){ if(phase!=Phase.PAID) throw new IllegalStateException("ship from "+phase); phase=Phase.SHIPPED; }
  void cancel(){ if(phase==Phase.SHIPPED) throw new IllegalStateException("cancel shipped"); phase=Phase.CANCELLED; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.state.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== State: order lifecycle ===");
    System.out.println("--- naive (illegal ship) ---");
var n=new NaiveOrder(); n.ship(); System.out.println(n.state);
System.out.println("--- state ---");
var o=new Order(); o.pay(); o.ship(); System.out.println(o.phase());
try { new Order().ship(); } catch (IllegalStateException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== State: order lifecycle ===
--- naive (illegal ship) ---
SHIPPED
--- state ---
SHIPPED
caught=ship from NEW
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with State.

Related Patterns

- Contrast State with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → State → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.